

PENSIONABLE.AI

Diorama

TECHNICAL OVERVIEW

EVERY ASSUMPTION EXPLICIT. EVERY CALCULATION TESTABLE.

How pensionable.ai turns pension rules, factors, spreadsheets, examples, and legacy calculation artefacts into structured, reviewable scheme-specific automation with Diorama.

MAY 2026 · EXTERNAL-FACING TECHNICAL PAPER

Diorama is the pension calculation engine from **pensionable.ai**.

This Is Not Another Pension Rules Engine

Most pension calculation platforms make one central promise:

We already have the standard functions. Bring your scheme, configure the tables, map the fields, and fit the rest into our model.

That model can be useful. It also creates the same frustration that experienced actuaries and calculation developers already know too well.

Even when a platform has reusable functions, standard factor logic, and configurable tables, the hard part does not go away. The team still has to:

- interpret messy scheme rules,
- reconcile conflicting source material,
- squeeze unusual benefit structures into someone else's abstraction,
- work around function boundaries that do not quite match the scheme,
- bolt on exceptions,
- hand-build validation,
- and prove that the final result really matches the scheme's intended behaviour.

In other words, the scheme still has to live by the system's rules.

The proposition behind **Diorama** is different.

We are not trying to force complex DB schemes into a fixed catalogue of standard functions and parameter tables.

We have built **Diorama**, an engine that handles hard GMP equalisation logic deterministically, with disciplined validation and external-witness reconciliation. Frontier models accelerate the structuring of messy scheme source material into candidates – typed rule objects, factor table extracts, methodology choices, provenance back to source – that the engine's validators and witness fixtures verify before deterministic compute runs on the money path.

Diorama does not generate scheme logic from prose. It implements approved-scope logic against structured rule inputs that humans review. The acceleration is in turning the messy front of the

project into structured candidates fast; the discipline is in keeping the money path explicit, validated, and witness-anchored.

That is the difference.

The Architectural Shift

This is the new paradigm: a modern pension engine should not just execute predefined pension logic.

It should help create new pension logic from the actual evidence pack:

- deeds and rules,
- administration guides,
- benefit specs,
- factor tables,
- spreadsheets,
- worked examples,
- calculation packs,
- prior reconciliations,
- operational notes,
- approved engagement decisions.

The old world says:

unstructured scheme material -> humans interpret everything ->
developers bespoke the system around it

The new world says:

unstructured scheme material -> engine-assisted extraction ->
structured rule basis -> generated scheme-specific functionality ->
deterministic compute -> replay and audit

That is not a cosmetic workflow improvement. It changes what the system is for.

Why Existing Systems Still Leave So Much Pain In Place

Traditional systems often get credit for the parts they have already standardised:

- retirement ages,
- table-driven revaluation,
- early and late retirement factors,
- commutation tables,
- tranche categories,
- pension increase tables,
- standard admin outputs.

But teams that actually implement complex schemes know that the real cost sits in the gaps between those standard components.

That is where people lose months:

- the rules do not quite fit the table shape,
- the date basis is more specific than the platform expected,
- the section/category logic cuts across the standard function set,
- GMP interacts with the rest of the benefit in a scheme-specific way,
- the exceptions matter just enough that they cannot be ignored,
- the examples prove that the “standard” path is not enough,
- the final implementation ends up as a pile of overrides and bespoke patches.

At that point, the platform has not removed the hard work. It has moved it into awkward, expensive corners.

What Is Different Here

Diorama starts from a different premise:

The source material is the raw input. The target is not a set of parameters. The target is the exact validated logic the scheme needs.

That means the engine is designed to do two things at once.

1. Carry a substantial built-in understanding of DB calculations

Diorama carries the disciplines a serious pension-logic engine needs:

- tranche-oriented modelling,
- factor and rate-table mechanics with explicit availability rules,
- validation disciplines around explicit assumptions and no-silent-defaults,
- replay and reconciliation patterns with documented tolerances,
- structured handling of the complex statutory mechanics in DB schemes – including the hard equalisation methods,
- a vocabulary for engine patterns that generalises across the full DB calculation surface.

That matters because we are not asking a model to invent pensions from scratch. We are giving it a strong domain frame to work inside.

2. Accelerate the build of missing scheme-specific functionality

Where the scheme does not fit the current exact feature set, Diorama does not have to stop at “unsupported.”

Instead, it uses frontier models to read and structure the scheme’s actual evidence pack and produce **candidates** – typed rule proposals, factor table extracts, methodology choices, provenance back to source, draft scaffolding for new schemas / validation logic / compute paths – that humans review against the source material and the engine’s existing disciplines.

The candidates are not the final answer. Approved extensions are then implemented and verified through the engine’s validators (rejecting incoherent construction at construction time) and witness fixtures (replay against worked examples with documented tolerances) before they enter the deterministic compute path.

That is the radical part – not that the engine generates production calculation logic from prose, but that it shrinks the time from “messy evidence pack” to “structured candidates ready for human review and engineering” from weeks to hours, while keeping the money path explicit, validated, and witness-anchored.

3. Deliver the result, not the tool

Pension calculation codebases at consultancies and admin houses have years of accreted convention – naming, factor-lookup patterns, tranche handling, validation idioms, audit-output formats. Replacing those conventions to adopt a new platform is the kind of friction that kills adoption no matter how good the new platform is.

We use Diorama. The consumer receives **artefacts** – Python modules, Excel proformas, regression harnesses, calculation packs, audit trails – in the consumer’s own code conventions.

Given representative examples of the existing codebase's structure, the workflow produces new scheme implementations in the same format. The consumer drops the code straight into its existing platform alongside every other scheme it already has. Its in-house team makes any subsequent changes using the skills they already have. The new scheme reads, runs, and maintains like every other scheme on the shelf.

This applies whether the consumer is a major consultancy with its own Python framework, an admin house with internal calculation patterns, or an in-house pension team with its own way of doing things. The convention stays the consumer's. The result is what changes hands.

What It Took To Get Here

The route to a pension-logic engine that handles real schemes, end-to-end, with auditable outputs is not obvious. A general-purpose AI tool will not give you one. A pension calculation platform you can buy off the shelf will not give you one either.

Diorama exists because of a specific combination of people and time.

- **Decades of pensions administration experience.** People who have spent careers inside the real mess – the deeds, the amendments, the data exceptions, the conventions that vary by scheme, the cases that survive every standardisation pass and still need answering.
- **People who have built pensions administration and calculation systems from scratch.** Not configured someone else's platform; built systems. Knowing what an engine actually has to do, and how the abstractions break under contact with real schemes, is different from knowing how to use one.
- **Frontier AI fluency since 2024.** Working with the latest generation of LLMs from the moment they became capable of structured technical work, learning where the model lifts the load and where it fails silently, building a workflow that respects both.

Each of these on its own gets you partway. The combination is what got us to a deterministic engine that can absorb messy scheme material, structure it under human direction, and produce auditable calculation outputs in weeks rather than months.

That combination is rare, and the work that gets you from "we have those people" to "we have a working engine that handles real schemes" is what makes Diorama unique.

What The Engine Actually Does With The Mess

The interesting question is not "can an LLM read a scheme booklet?" That is the least important part of the story.

The interesting question is:

How do you get from a pile of inconsistent pension artefacts to a database-backed calculation capability that produces auditable results fast enough to change the economics of scheme implementation?

The answer is a staged engine workflow that we can execute now.

Stage 1: Devour the evidence pack

Diorama takes the messy estate as input:

- rule documents,
- amendments,
- member guides,
- spreadsheets,
- factors,
- prior calculations,
- workbook extracts,
- examples,
- notes from prior reconciliations.

The point is not to pretend those documents are clean. The point is to ingest the unstructured mess as it really exists.

Stage 2: Use frontier models to convert mess into candidate structure

This is where LLMs matter.

They are used to read the prose, tables, headings, matrices, workbook shapes, and documentary fragments and produce structured candidates for:

- rule areas,
- field values,
- factor references,
- date conventions,
- methodology choices,
- known exclusions,
- possible ambiguities,
- provenance back to where the material came from.

This is a huge acceleration point. It cuts down the manual labour of turning pension text and spreadsheets into something an engine can work with.

Stage 3: Use engine knowledge to constrain and validate

The raw model output is not the final answer.

Diorama already knows what coherent pension logic looks like. That means it can subject extracted candidates to strong checks:

- required fields,
- coherent date logic,
- valid factor coverage,
- explicit methodology choices,
- all-or-nothing comparator requirements,
- unsupported feature flags,
- contradictions across source material.

This is where the combination matters. Frontier models provide speed and breadth. The engine provides discipline.

Stage 4: Create the exact scheme logic

Once the basis is explicit and approved, Diorama can create the scheme-specific functionality itself:

- the structured calculation basis,
- the required validation rules,
- the factor-table bindings,
- the projection or comparison logic,
- the calculation code path,
- the test fixtures,
- the reconciliation harness,
- the audit output.

This is the part standard systems do not do well. They ask the team to adapt the scheme to the product.

This engine adapts itself to the scheme.

Stage 5: Produce deterministic results fast

After the scheme logic exists, the runtime should be deliberately boring:

- no source-document reading,
- no hidden interpretation,
- no runtime guesswork,
- no LLM in the money path.

Just structured inputs, deterministic compute, replay against examples, and auditable outputs.

That is how we get both speed and control.

Trust And Safety: How We Make This Defensible

The engine is fast. The question every serious buyer asks next is: how do you make this safe for actuarial work, defensible for trustees, and compliant with GDPR.

Five answers, each a structural design choice rather than a policy promise.

1. The actuary owns the deliverable. The human in the loop directs the engagement throughout – frames the brief, makes the actuarial judgment calls, decides which interpretive readings to adopt, names which placeholders the data team owns versus which the engagement team must answer, and signs off the answer. The engine and the LLMs are tools that accelerate execution and review; the actuary owns the calculation and the conclusions drawn from it. The accountability never moves.

2. No LLM in the runtime path. Diorama is built with LLM assistance during development. The runtime is deterministic Python and Excel formulas. The same code produces the same outputs every time. The LLM helps build the calculator; it does not enter the audit trail of any individual calculation. That is the deterministic-infrastructure principle, and it is the difference between “an AI did this calculation” – which actuaries cannot sign off – and “AI-assisted developers built deterministic software that did this calculation” – which works the same way every other piece of audited software does.

3. Every output testable. The calculator’s formulas are mechanical translations of the specification’s rule text. A regression harness re-implements each rule in Python independently and asserts the workbook’s output matches. The harness runs on every rebuild. Any drift between rule text, workbook formula, and harness reference implementation surfaces as a test failure before anything ships. The harness is the quality gate – a structural answer to “how do we know this is right?”.

4. Two independent LLMs working in tandem to get to the truth. A development LLM does the execution; a review LLM from a different model family by a different vendor audits the output, operating from the same source artefacts with no shared context. The two work in tandem – the reviewer surfaces issues the execution model missed, the execution model addresses them, and

successive rounds tighten the work until both converge on the same answer. AI-on-AI peer review is structurally analogous to human peer review and catches the things one model alone tends to miss. The execution model only sees the reviewer's findings as written recommendations to act on.

5. Anonymisation before any LLM access. For engagements where source artefacts contain personal data, anonymisation moves into the pipeline itself. Six phases: structure-only extraction (the LLM-driven classification step sees only sheet names, column headers, named ranges, inferred types – never cell values), header classification, human-in-the-loop confirmation, deterministic substitution, validation against known PII regex patterns, and only then LLM access to the anonymised artefact. The architectural lever is **separation of “knowing what to do” from “doing it”** – the classifier sees only metadata and cannot physically access member data. The substitution map is held locally and never transmitted. The audit trail is available to the client's data protection officer on request.

Each of these is structural, not procedural. They are properties of how the engine is built, not promises about how careful the team is.

Why This Is Exciting For Actuaries

An actuary should read this and think:

Finally, a system that is not asking me to flatten the scheme into a generic box and hope the exceptions survive.

The exciting part is not only faster implementation. It is that the engine can make the real rule shape visible.

That means:

- fewer hidden assumptions,
- less manual translation from prose to code,
- clearer challenge points,
- better use of worked examples,
- easier review of what has actually been implemented,
- faster route from rule understanding to tested outputs.

Why This Is Exciting For Developers

A developer used to coding DB calculations into legacy systems should read this and think:

We could stop spending months building bespoke infrastructure around every awkward scheme and instead use an engine that already knows the domain, already knows GMP, and can help generate the missing scheme- specific functionality from the evidence pack itself.

That is a completely different development model.

Instead of spending most of the project on:

- interpretation plumbing,
- spreadsheet archaeology,
- factor glue,
- exception wiring,
- hand-built test scaffolding,
- after-the-fact explanation,

the team can spend more of its time on:

- reviewing real rule choices,
- extending genuine domain capability,
- refining generated logic,
- tightening replay evidence,
- closing the last hard scheme-specific gaps.

The Bigger Point

The opportunity is to radically accelerate all pensions calculation work:

- start from the real evidence pack,
- let frontier models turn it into structured candidates fast,
- let humans review and approve the basis,
- let the engine implement and verify approved-scope logic against validators and witness fixtures,
- keep deterministic control over the money path,
- carry the audit trail inside the work – every formula traceable to spec, every output testable, no LLM in the runtime path,
- and reach working, auditable calculation outputs far faster than the traditional bespoke route.

That is what should make technical readers sit up.

Diorama absorbs the mess, holds the disciplines that make hard pension logic safe to run, and shrinks the path from messy evidence pack to structured, validated, witness-anchored scheme-specific capability. This document is framed around DB work because that is where the complexity lives; DC and CARE schemes are structurally simpler, and the same engine applies.